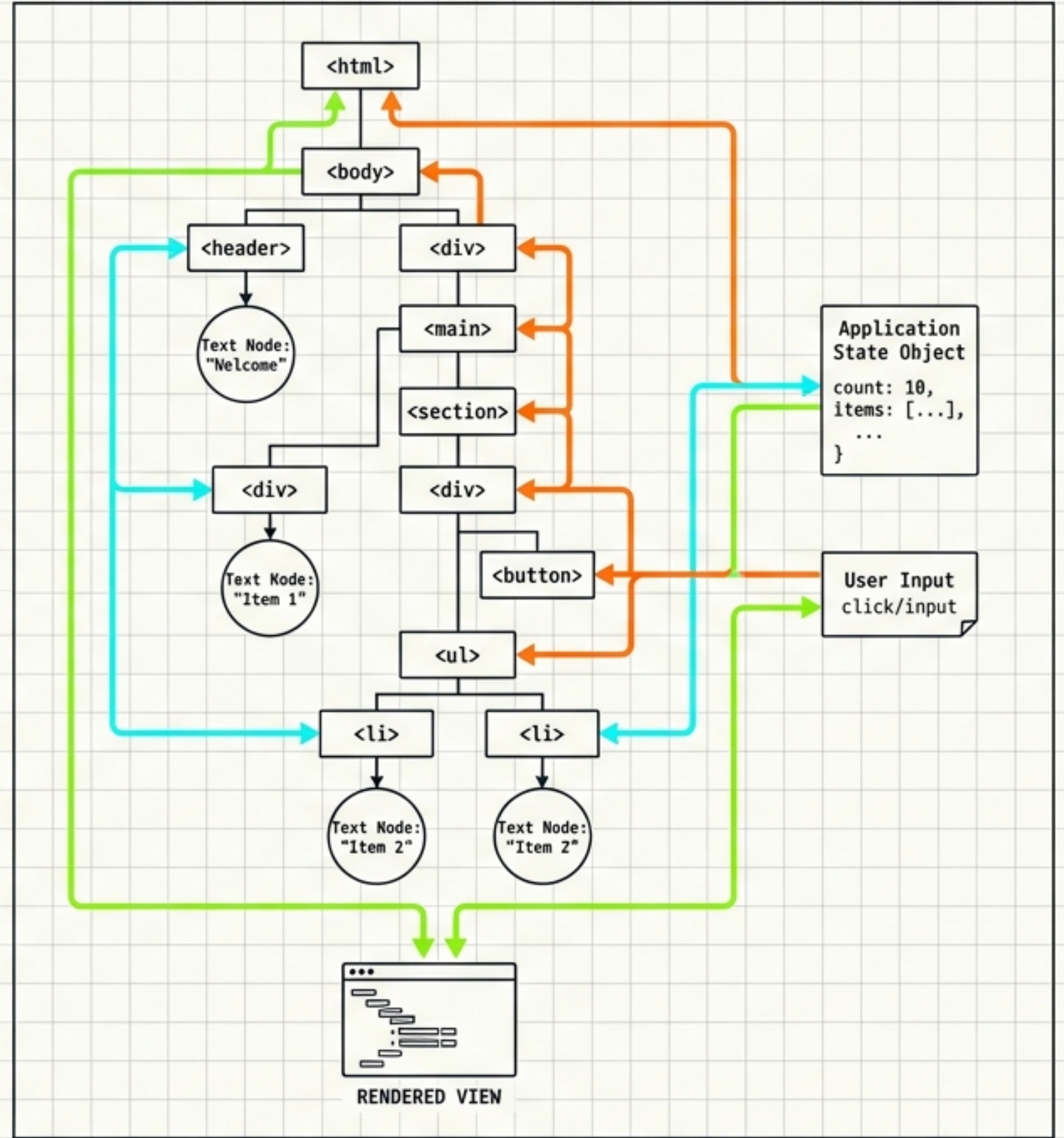


바닐라 JS로 정복하는 웹 렌더링과 이벤트 흐름

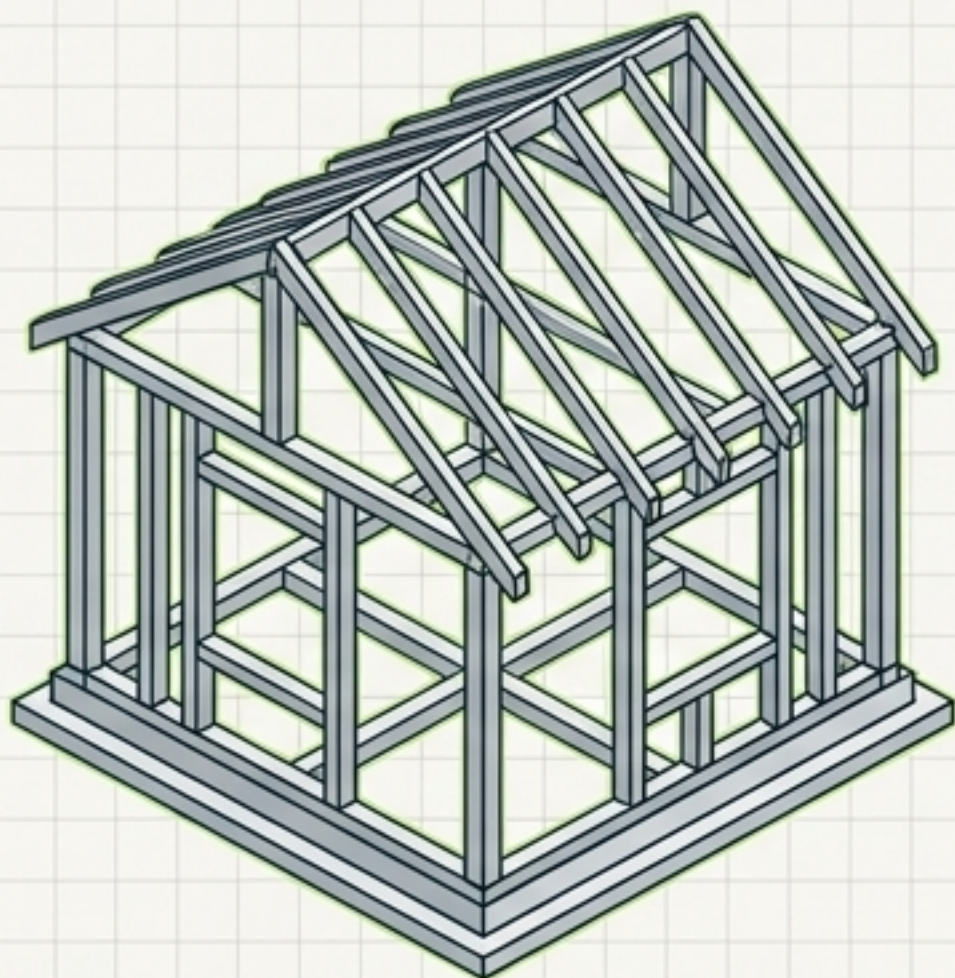
Event → State → Render

아키텍처 딥다이브



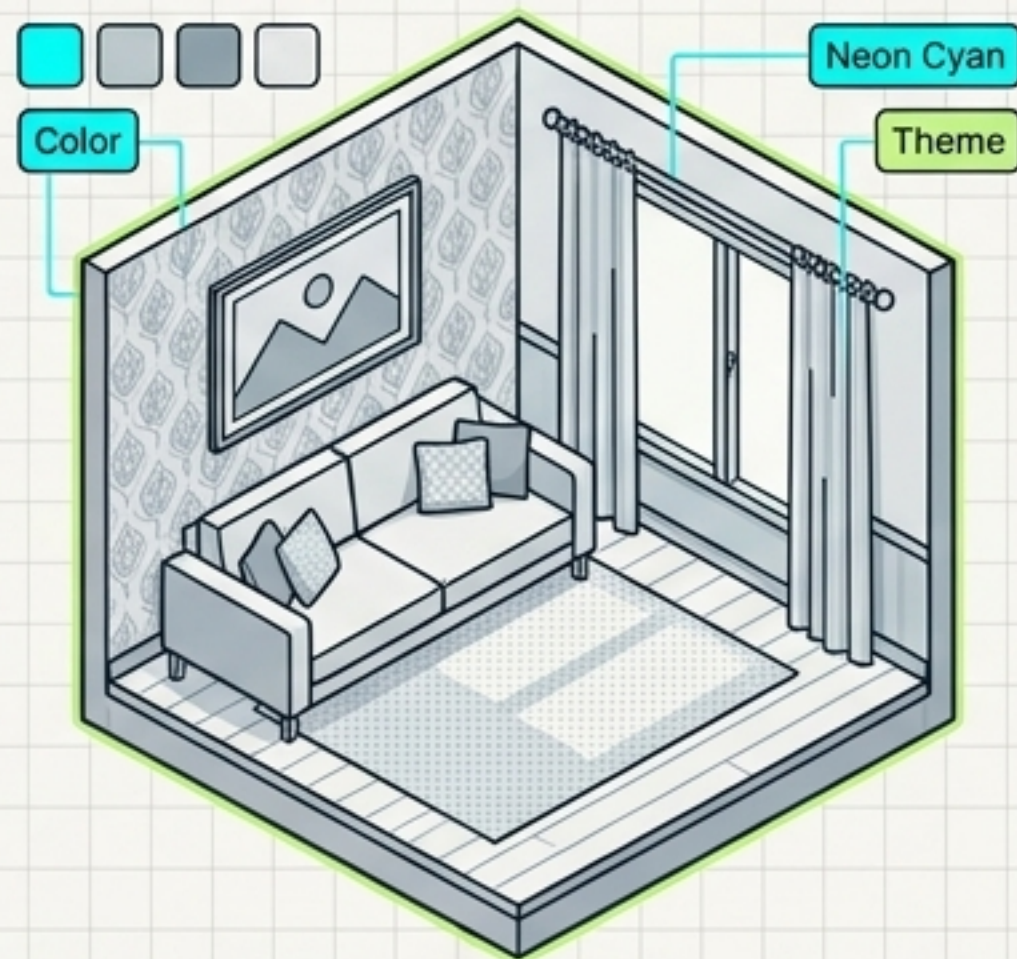
웹의 3대 요소와 역할 분담

뼈대 (HTML)



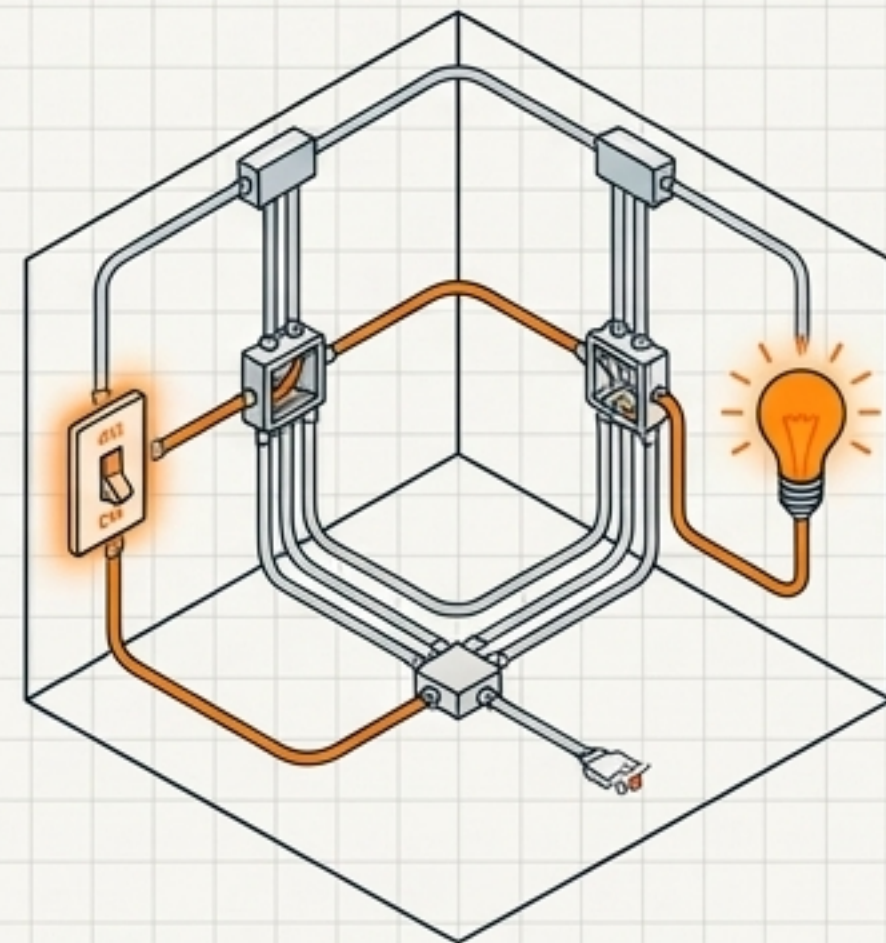
- 시맨틱 구조
- 검색 엔진 최적화 (SEO)
- 웹 접근성

인테리어 (CSS)



- 시각적 디자인 토큰 (:root)
- 반응형 레이아웃 (Flex/Grid)
- 다크/라이트 테마

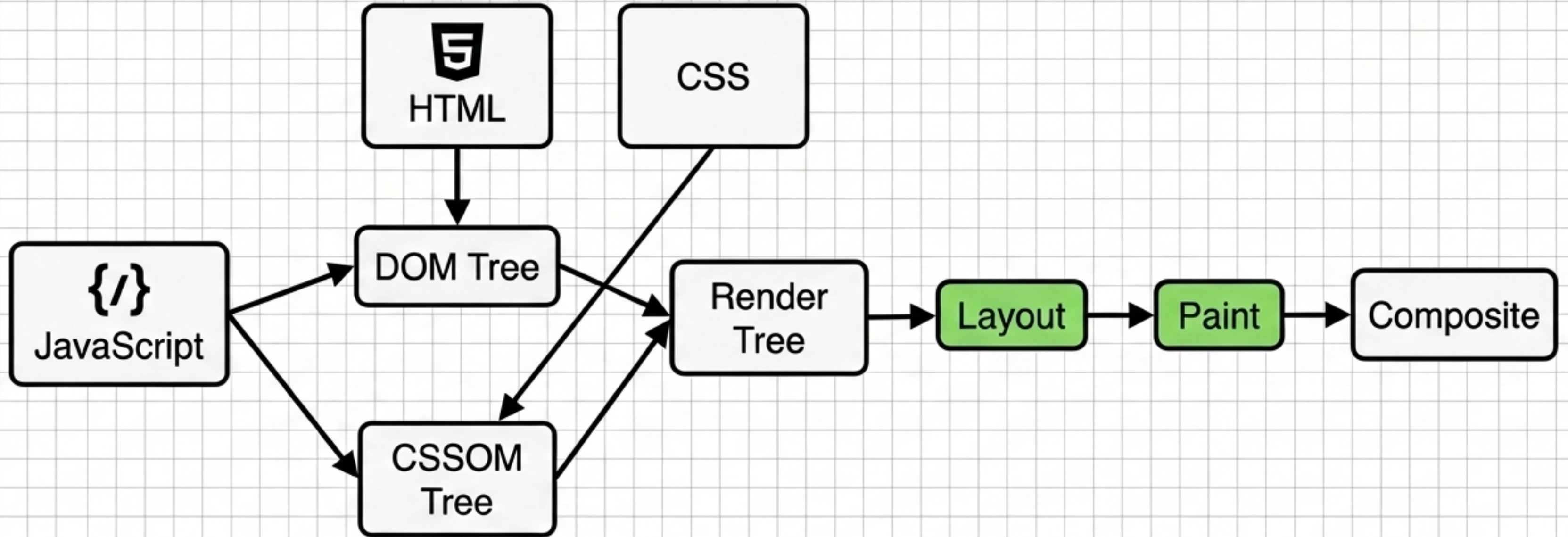
전기 회로 (JavaScript)



- 동적 인터랙션 (이벤트)
- 상태(State) 데이터 제어
- 비동기 서버 통신

관심사의 분리 (Separation of Concerns): 코드가 얹히지 않게 역할을 분리하여 유지보수성, 브라우저 캐싱 성능, 프레임워크 확장성을 극대화합니다

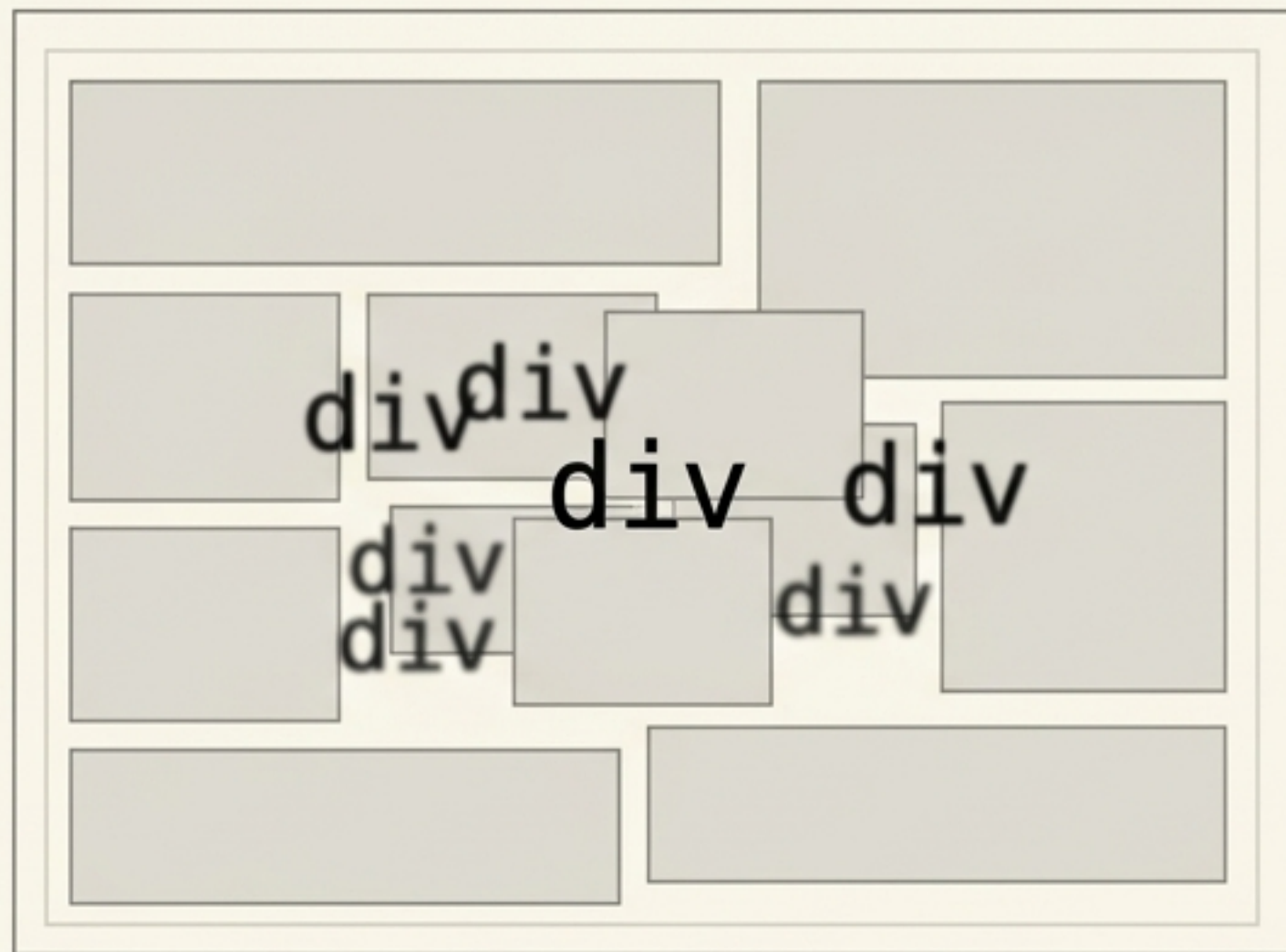
브라우저의 렌더링 파이프라인 (Critical Rendering Path)



핵심 인사이트: 렌더링 과정을 이해해야 JavaScript로 DOM/CSSOM을 조작할 때 발생하는 값비싼 성능 저하(Reflow, Repaint)를 최소화하는 최적의 코드를 작성할 수 있습니다.

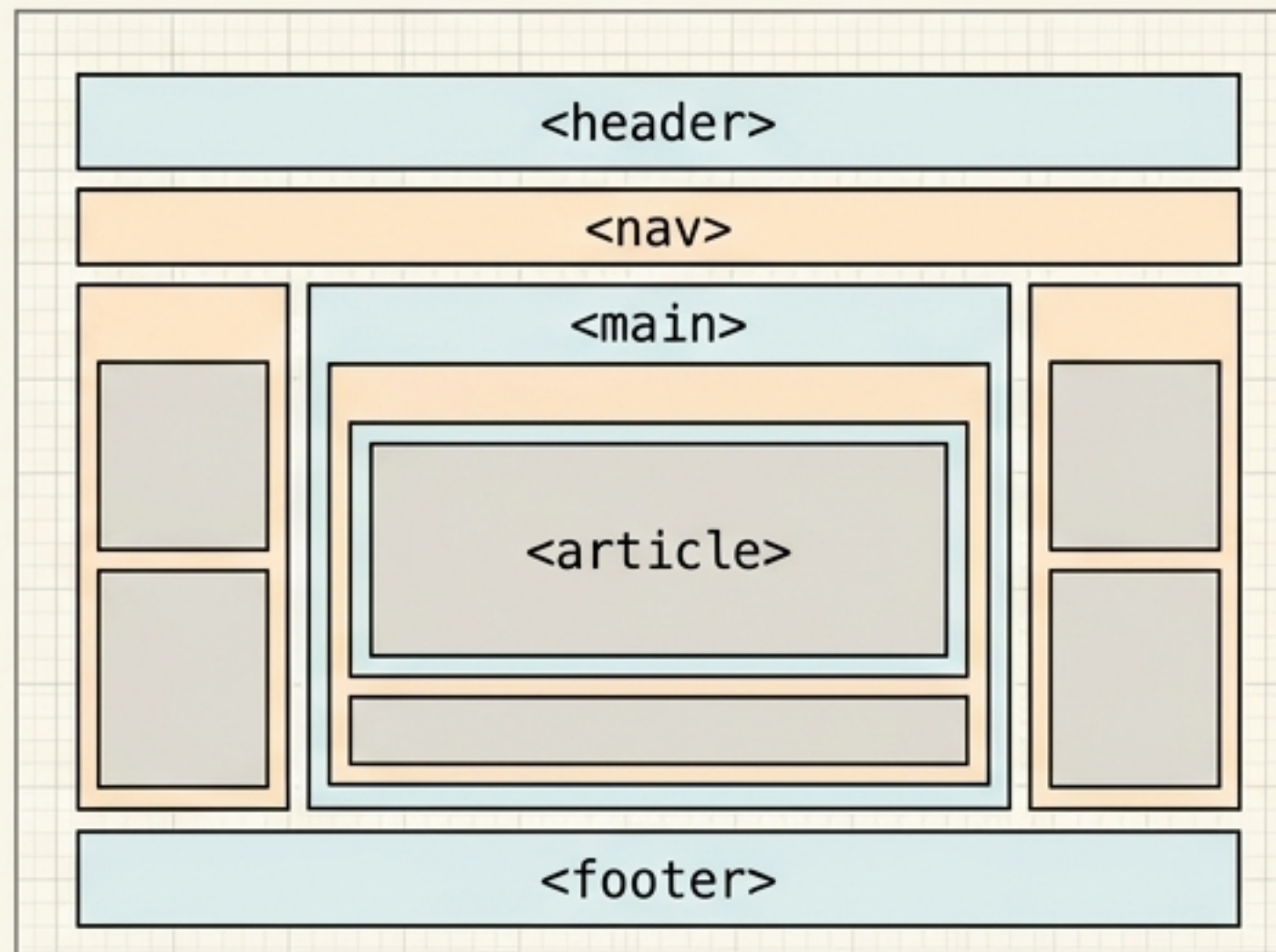
시맨틱 DOM: 기계가 코드를 이해하는 엑스레이 비유

비시맨틱 구조 (div 지옥)



검색 엔진과 스크린 리더가 영역의 우선순위와 목적을 전혀 파악할 수 없음.

시맨틱 구조 (의미론적 설계)



로봇이 문서의 뼈대를 정확히 색인 (SEO 최적화)하고, 스크린 리더 사용자가 랜드마크를 따라 쉽게 페이지를 탐색 (접근성 향상).

유지보수성: 다른 개발자가 코드를 보았을 때 영역의 역할을 즉각적으로 인지할 수 있습니다.

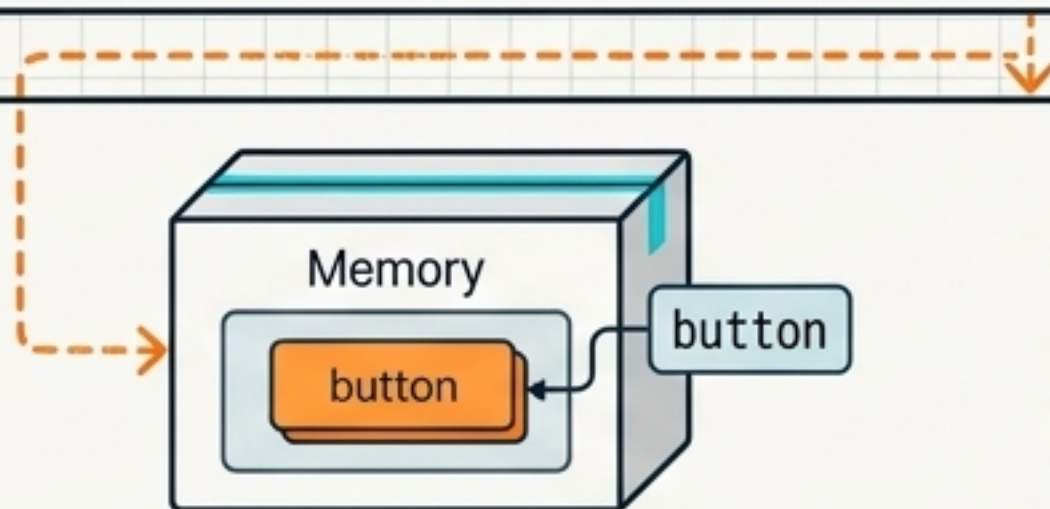
상호작용의 3단계: 찾기, 모으기, 연결하기



1. Select (찾기)

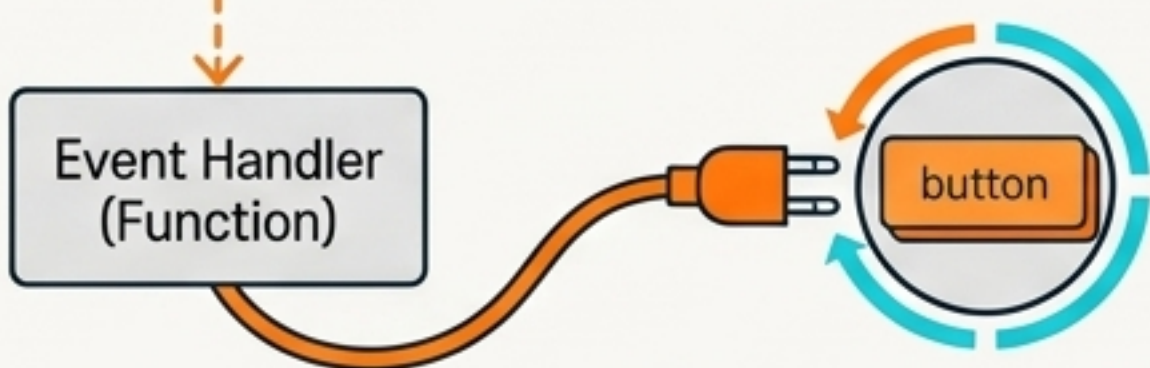
querySelector: DOM 트리에서 상호작용할 요소를 정밀하게 탐색

```
const button = document.querySelector('.btn');
```



2. Cache (모으기)

변수 할당: 찾은 요소를 메모리에 캐싱하여 불필요한 반복 탐색 방지



3. Bind (연결하기)

addEventListener: 특정 이벤트(Click, Scroll)와 실행할 로직(함수)을 안전하게 결합

```
button.addEventListener('click', handleClick);
```



안티 패턴:
인라인 onclick 속성

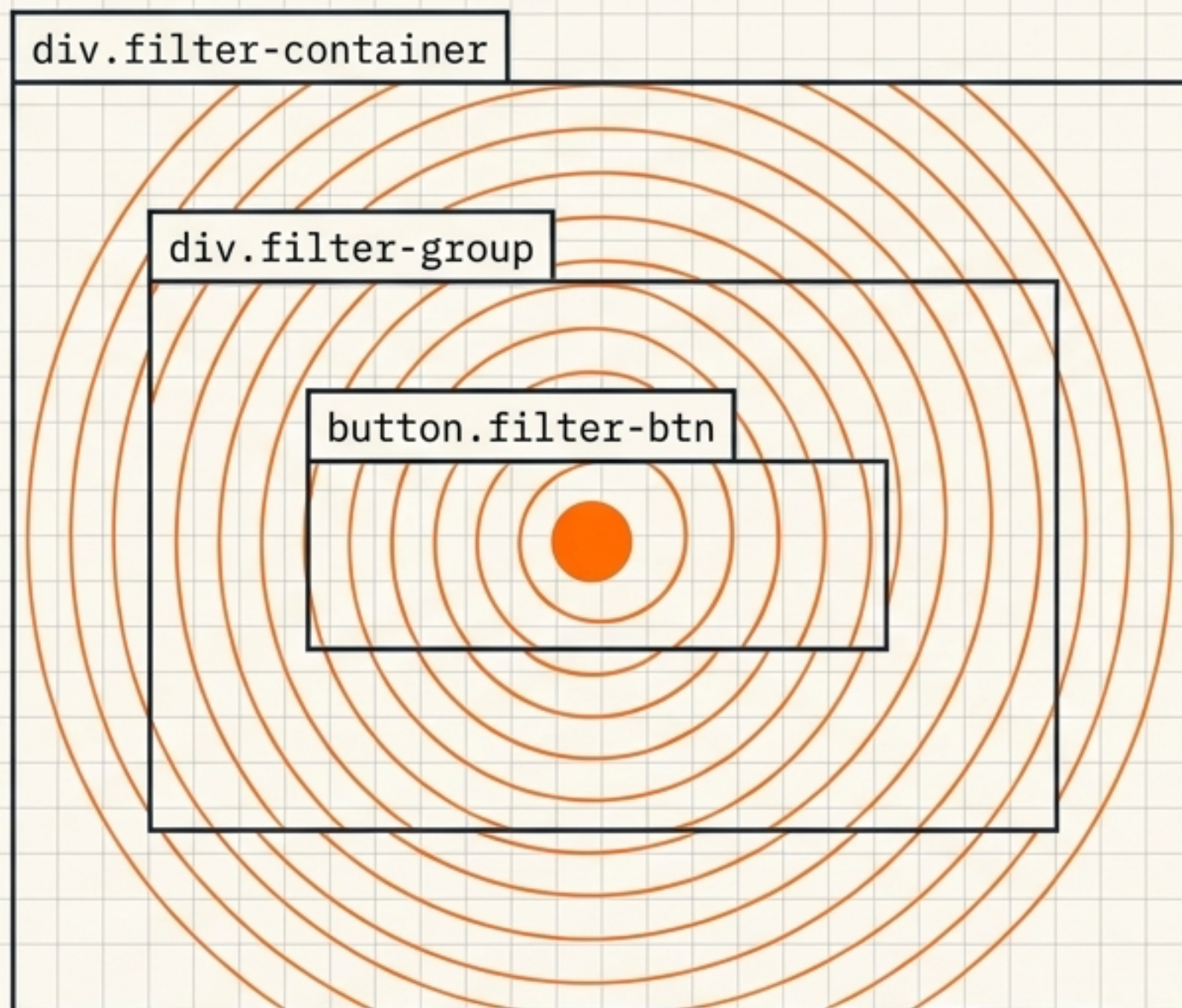
HTML 구조와 JS 로직이 강하게 결합되어 코드가 지저분해지고 유지보수가 불가능해짐.



모던 패턴:
addEventListener

구조와 로직의 완벽한 분리.
동적으로 이벤트를 추가/해제하기 용이함.

이벤트 감지와 위임 (Event Delegation)



문제점 (각각 연결)

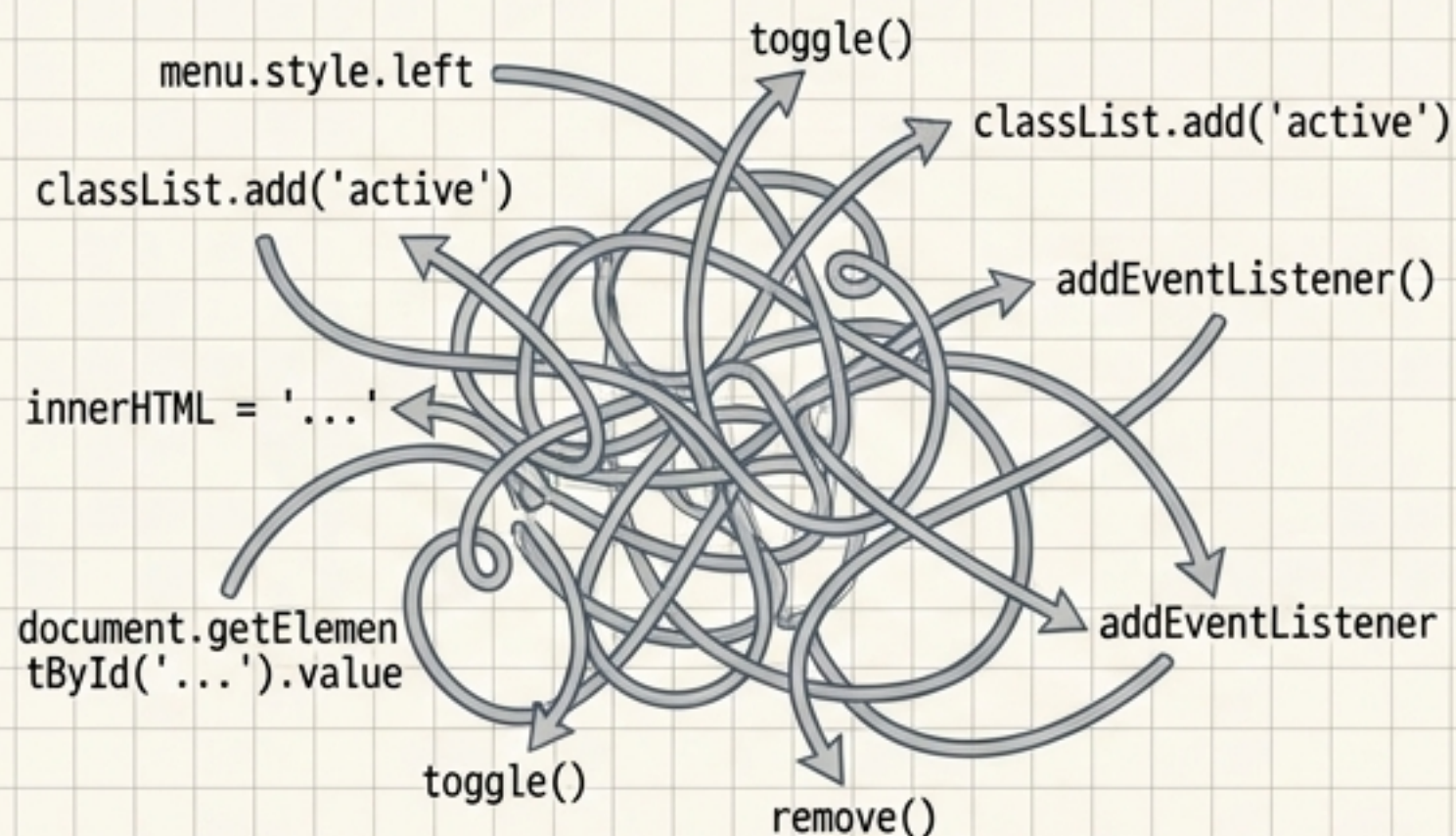
수십 개의 버튼마다 개별적으로 이벤트 리스너를 달면 메모리 낭비가 심하고, 나중에 동적으로 추가된 버튼은 이벤트를 듣지 못합니다.

해결책 (이벤트 버블링과 위임)

상위 부모 컨테이너에 단 하나의 리스너만 등록합니다. 클릭 이벤트가 물결처럼 퍼져나가는 버블링 특성을 활용해, `e.target`으로 실제 클릭된 자식 요소를 식별합니다. 성능과 확장성을 동시에 잡는 고급 패턴입니다.

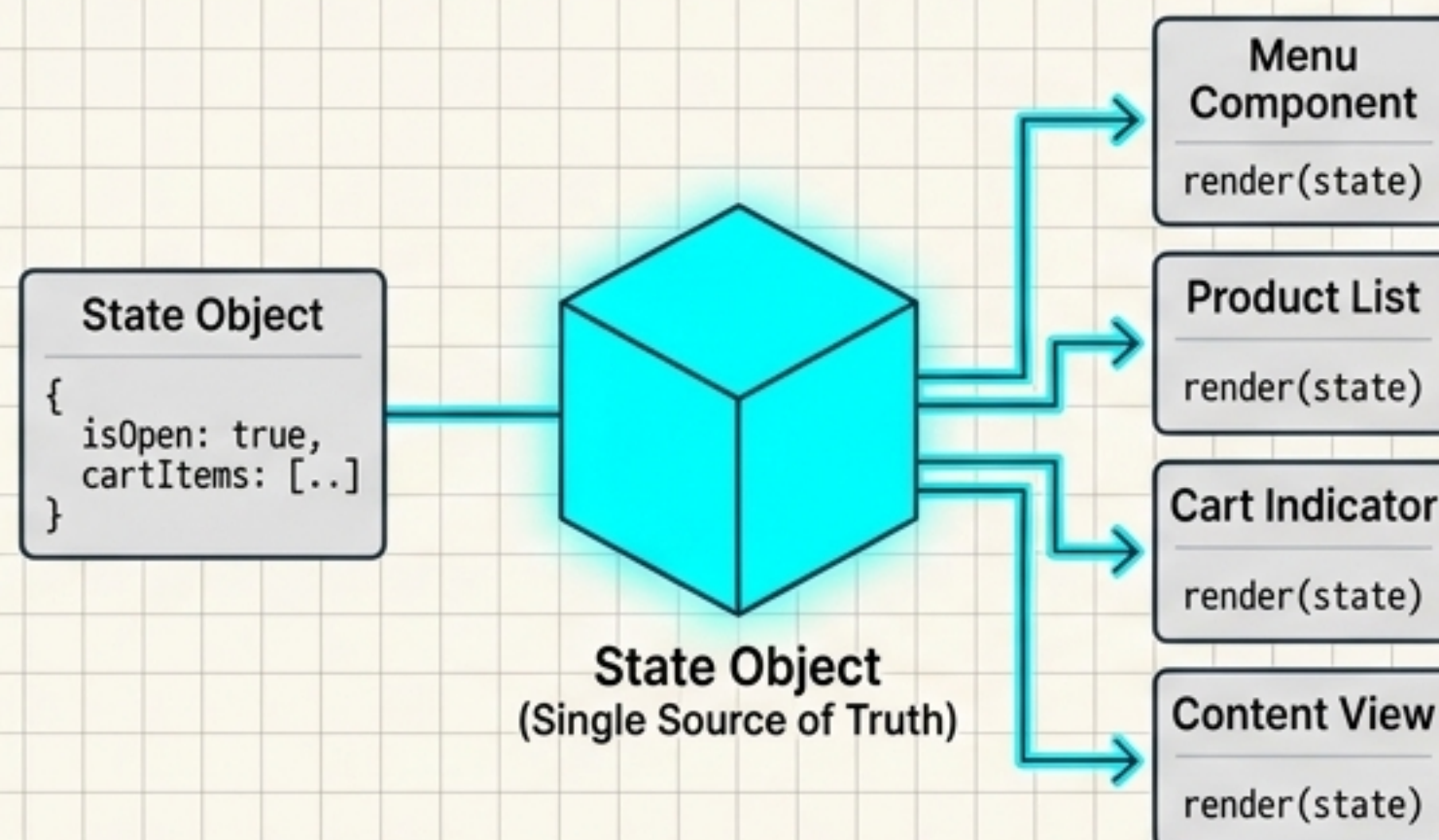
패러다임 전환: 스파게티 코드 vs 상태 기반 구조

직접 DOM 조작 (과거)



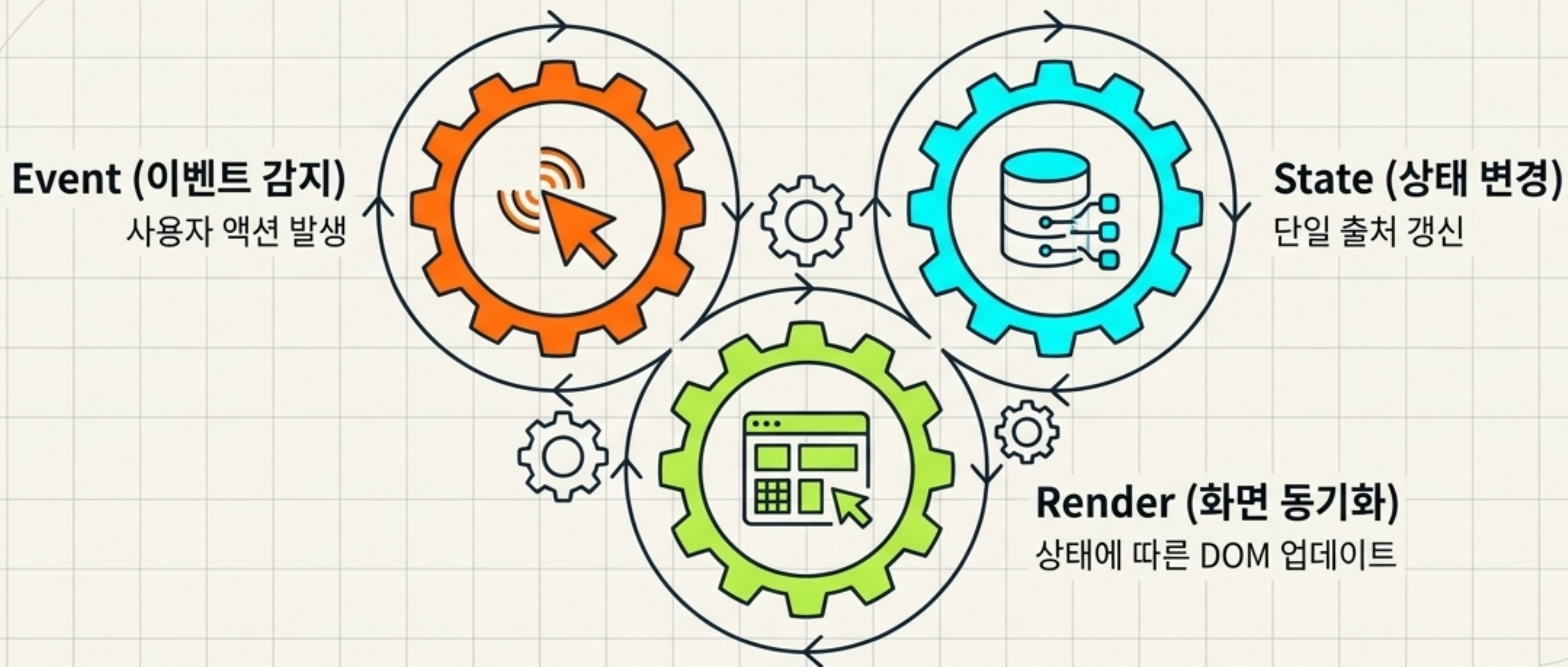
상태가 DOM 요소 곳곳에 파편화되어 흩어져 있습니다. 앱이 복잡해지면 '현재 메뉴가 열려있는가?'와 같은 현재 상태를 추적하는 것이 불가능해지며 버그의 온상이 됩니다.

상태 기반 구조 (모던)



앱의 모든 핵심 데이터를 하나의 '상태 객체'에 집중시킵니다. 상태 값만 변경하면 화면은 그 상태를 거울처럼 투영하여 자동으로 동기화됩니다. 디버깅과 유지보수가 압도적으로 쉬워집니다.

모던 프론트엔드의 핵심 메커니즘



단방향 데이터 흐름 (Unidirectional Data Flow): 화면(DOM)은 오직 상태(State)를 거울처럼 투영할 뿐, 스스로를 직접 변경하지 않습니다. 이것이 바닐라 JS를 넘어 React, Vue 등 모든 모던 프레임워크를 관통하는 가장 위대한 패러다임입니다.

구현 해부학 1: 동기적 렌더링 (다크 모드)

동기적 렌더링 로직

1. 이벤트 (Event)

```
btn.addEventListener('click', ...)
```

⇓

2. 상태 변경 (State)

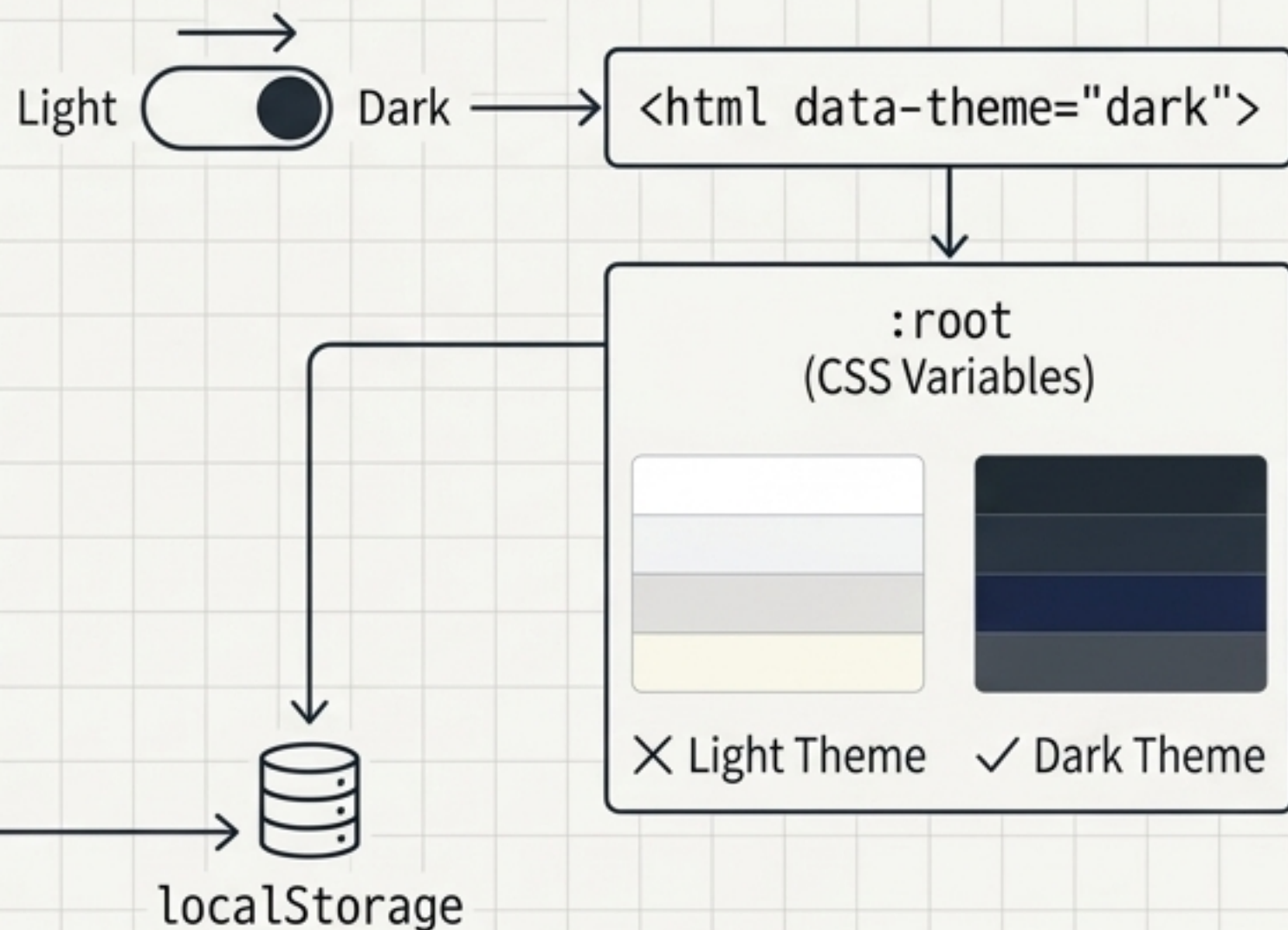
```
state.theme = state.theme === 'light' ?  
'dark' : 'light'
```

⇓

3. 화면 업데이트 (Render)

```
const { theme } = state;  
renderTheme()
```

UI 메커니즘 다이어그램



구조분해 할당(Destructuring)으로 코드를 간결하게 묶고, 상태 변화에 맞춰 CSS 변수(:root)를 일괄 전환하는 기술적 우아함을 보여줍니다.

비동기 렌더링의 흐름: Fetch & Async/Await



fetch API

비동기로 서버에 데이터를 요청하는 브라우저 내장 네트워크 기능.

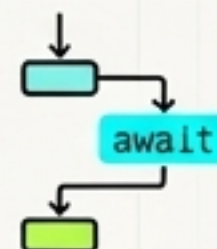
```
fetch('https://api.example.com/data')
```



async / await

콜백 지옥(Callback Hell)을 벗어나, 복잡한 비동기 코드를 동기적 코드처럼 직관적이고 순차적으로 읽히게 만드는 최신 자바스크립트 문법.

```
async function getData() {  
  const response = await fetch(...);  
}
```



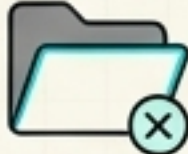
try / catch

네트워크 오류나 서버 속도 제한(Rate Limit 403) 등의 예외 상황을 안전하게 포착하고 튼튼하게 방어하는 필수 에러 처리 구문.

```
try {  
  ...  
} catch (error) {  
  console.error(error);  
}
```

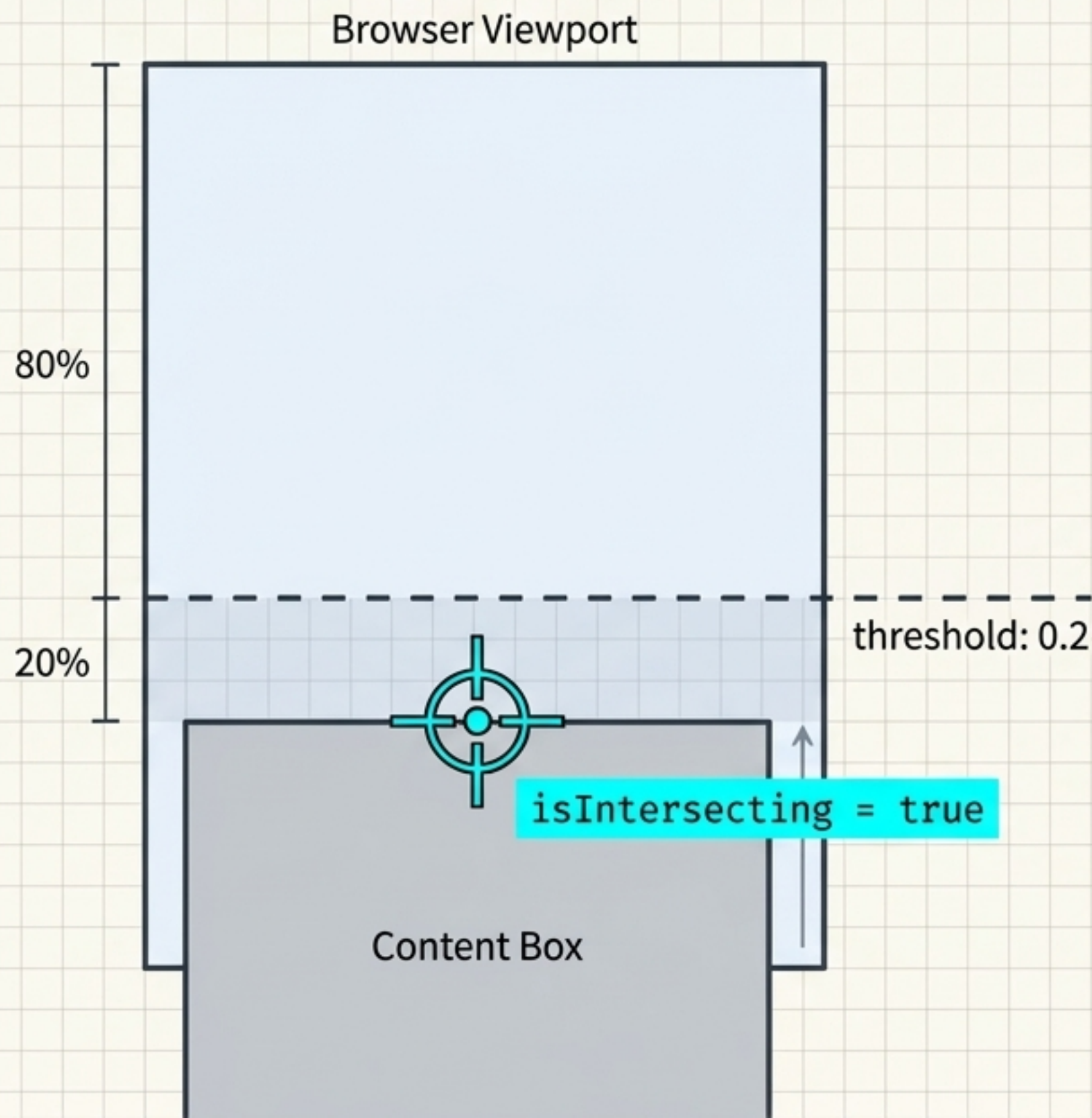


구현 해부학 2: 비동기 4가지 상태 UI (GitHub API)

apiStatus: 'loading'	apiStatus: 'success'
 데이터를 불러오는 중입니다...	 Project Name Description of the repository... Last updated... View  Project Name Description of the repository... Last updated... View  Project Name Description of the repository... Last updated... View
apiStatus: 'error'	apiStatus: 'empty'
 불러올 수 없습니다 [재시도] ↺	 표시할 데이터가 없습니다

완벽한 UX 설계: 비동기 통신 시 백그라운드에서 무슨 일이 벌어지는지 사용자에게 완벽하게 피드백을 제공해야 합니다. 상태 값(apiStatus) 하나에 따라 renderProjects()가 템플릿 리터럴 마크업을 동적으로 생성합니다.

DOM 렌더링 성능 최적화: Intersection Observer



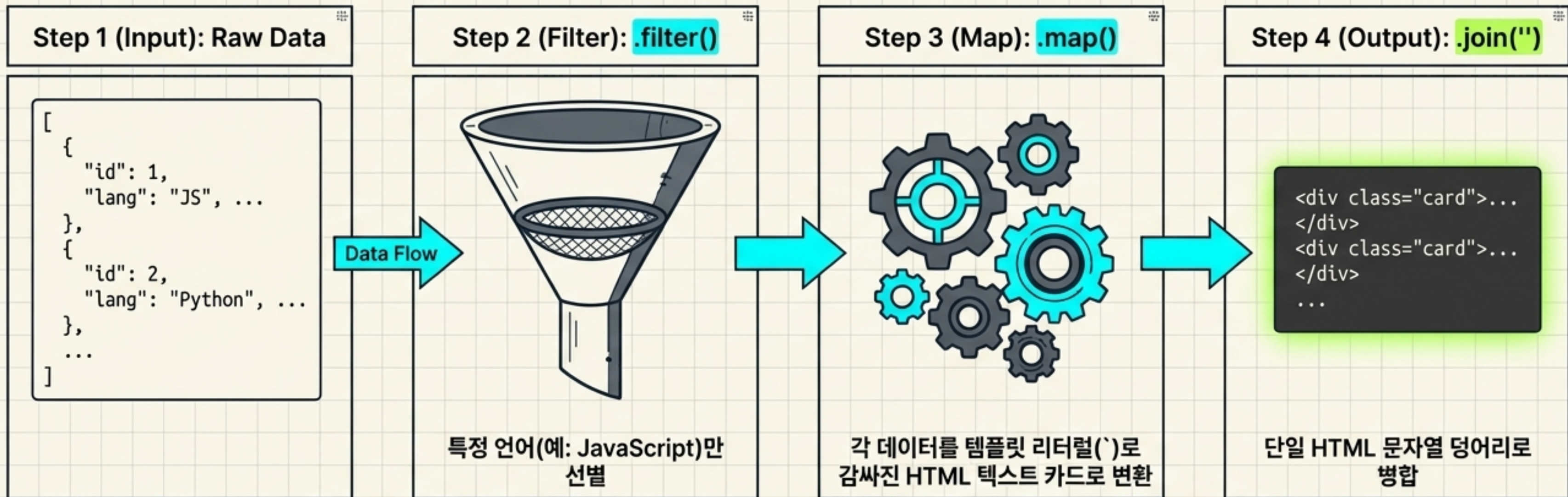
- 무거운 스크롤 이벤트 회피

스크롤 이벤트 리스너를 통해 픽셀 위치를 직접 매 프레임마다 계산하면 심각한 성능 저하가 발생합니다.

- 브라우저 내장 API 활용

Intersection Observer를 사용하여 요소가 뷰포트에 걸치는 순간만 감지해 스크롤 애니메이션(**fade-in**)을 최적화합니다. 감지 후에는 **unobserve**로 즉시 자원을 해제하여 메모리를 절약합니다.

모던 JS(ES6+)의 데이터 변환 파이프라인



ES6+ 문법의 강력함: 구조분해 할당, 화살표 함수, 그리고 배열 메서드가 결합되면 복잡한 for 루프 없이 선언적이고 간결하게 '상태 데이터'를 'DOM 문자열'로 가공할 수 있습니다.

결론: 바닐라 JS에서 React로 (The Bridge)

Vanilla JS 아키텍처

1. "state 객체"

2. "render() 함수 호출"

3. "document.setAttribute
(실제 DOM 제어)"

React 아키텍처

1. "useState 혹"

2. "컴포넌트 재실행 및 가상
DOM(Virtual DOM) 비교"

3. "변경된 실제 DOM 자동
렌더 업데이트"

상태 관리 (State Management)

렌더링 로직 (Rendering Logic)

DOM 조작 (DOM Manipulation)

왜 프레임워크 이전에 순수 JS 원리를 알아야 할까요?

오늘 체득한 '이벤트 → 상태 → 렌더링' 패턴은 모든 모던 웹 프레임워크의 숨겨진 엔진 그 자체입니다.
근본적인 뼈대를 완벽히 이해한 자만이 더 높고 견고한 소프트웨어 건물을 세울 수 있습니다.